RNSG: A Range-Aware Graph Index for Efficient Range-Filtered Approximate Nearest Neighbor Search

Abstract

Range-filtered approximate nearest neighbor (RFANN) search is a fundamental operation in multimedia and data retrieval. Given a set of objects, each with a vector embedding and a numerical attribute, an RFANN query retrieves the nearest neighbors to a query vector among those objects whose numerical attributes fall within a specified range. State-of-the-art methods for RFANN search often require constructing multiple range-specific graph indexes to achieve high query performance, which incurs significant indexing overheads. To address this, we first establish a novel graph indexing theory, the range-aware relative neighborhood graph (RRNG), which jointly considers spatial and attribute proximity. We prove that the RRNG satisfies two crucial properties: (1) *monotonic searchability*, which ensures correct nearest neighbor retrieval via beam search; and (2) *structural heredity*, which guarantees that any range-induced subgraph remains a valid RRNG, thus enabling efficient search with a single graph index. Based on this theoretical foundation, we propose a new graph index, called RNSG, as a practical solution that approximates RRNG efficiently. We develop fast algorithms for both constructing the RNSG index and processing RFANN queries with it. Extensive experiments on five real-world datasets show that RNSG achieves significantly higher query performance with a more compact index and lower construction cost than existing state-of-the-art methods.

1 Introduction

Nearest neighbor search in high-dimensional Euclidean spaces is a critical task for applications ranging from information retrieval [31] and database systems [21, 45, 49] to generative artificial intelligence [53]. However, the curse of dimensionality [6, 23, 48] poses a fundamental challenge to traditional exact search methods, rendering them prohibitively slow for practical use. To overcome this, approximate nearest neighbor (ANN) search has emerged as a practical alternative, trading a marginal loss in accuracy for orders-of-magnitude gains in query efficiency.

Over the past decades, numerous ANN search techniques have been developed, including tree-based [1, 7, 42], hashing-based [44, 46], quantization-based [17, 18, 20, 24], and graph-based methods [15, 16, 22, 32, 33, 47]. Comprehensive benchmarks [5, 47] have established graph-based methods as the current state-of-the-art in terms of both query efficiency and accuracy. These techniques index data into a graph where nodes correspond to data objects and edges are established based on proximity. A beam search algorithm [16, 34, 46] is then used for query processing. A key insight is that the top-performing graph indexes, including HNSW [33] and NSG [16], are engineered as approximations of the Relative Neighborhood Graph (RNG) [43]. The RNG supports these methods with its desirable theoretical search guarantees and robust practical performance (see Section 2 for details).

Recent research has expanded to more complex ANN query variants to better serve real-world applications [8, 19, 54]. A prominent example is the Range-Filtered Approximate Nearest Neighbor (RFANN) search, which has attracted significant interests [30, 38, 50, 51, 54]. In this setting, each data object comprises a vector and a totally-ordered numeric attribute. An RFANN query takes a query vector and a range constraint on the numeric attribute, returning the nearest neighbors from the set of objects that satisfy the range filter. The RFANN query supports numerous practical applications. For example, on an e-Commerce platform, a user can search for products by a text description (encoded as a vector) while restricting results to a specific price range. Similarly, in Apple's on-device photo management system [41], users can retrieve visually similar photos taken within a given date range, where both data images and query images are encoded into vectors to prevent personal information leakage, thereby meeting personalized needs efficiently while preserving data privacy.

Three basic strategies exist for RFANN queries [45]: pre-filtering, in-filtering, and post-filtering. These approaches share the characteristic of executing searches on a standard ANN index, differing primarily in when the range filter is applied. Specifically: (1) pre-filtering first selects in-range objects, then performs ANN search on this subset [45]; (2) in-filtering checks the range condition during the ANN search process [12], skipping out-of-range objects; and (3) post-filtering conducts a standard ANN search first, then filters the results by range [27]. The fundamental limitation of all these basic strategies is their inability to adapt efficiently to diverse query workloads with varying range selectivities. Although some hybrid methods attempt to dynamically combine these strategies [45], they remain constrained by the same underlying limitations, inevitably yielding suboptimal performance.

Recent state-of-the-art methods directly integrate ANN graph indexes with numeric attributes [50, 54], yet still struggle to balance efficiency and effectiveness. For instance, [54] proposes building a dedicated ANN index for every possible query range. While this achieves high performance, it incurs prohibitive indexing costs. A subsequent compression technique [54] reduces this overhead by constructing only multiple representative indexes, but at the cost of significantly degraded query accuracy. Similarly, [50] integrates an ANN graph index with a *segment tree* [9], building indexes for a moderate number of ranges; this improves practicality but still entails substantial indexing overhead.

The fundamental difficulty in balancing index efficiency and query performance originates from a structural mismatch: while existing graph-based ANN indexes are designed based on spatial proximity, range filtering inherently disrupts these spatial relationships. For instance, RNG-based indexes prune edges based on spatial relationships, but when a query range filters out nodes, the remaining graph often suffers from broken connectivity and poor searchability, causing significant performance degradation (see Fig. 1).

To overcome these limitations, we propose a novel graph indexing theory for RFANN queries: the Range-aware Relative Neighborhood Graph (RRNG). Unlike traditional graph indexes that consider only spatial proximity, the RRNG jointly considers spatial and attribute proximity. This fundamental enhancement enables the RRNG to guarantee two crucial properties: (1) *monotonic searchability*, preserving the RNG's guarantee that beam search will correctly locate

SIGMOD '26, Bengaluru, India
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/XXXXXXX.XXXXXXX

nearest neighbors of any node, and (2) *structural heredity*, ensuring any range-induced subgraph maintains the RRNG properties (a capability the standard RNG fundamentally lacks), thus providing consistent search performance across arbitrary query ranges. Moreover, we prove that the RRNG incurs only a modest logarithmic overhead in both index size and search complexity compared to the traditional RNG. These theoretical advantages directly address the fragmentation problem in traditional graph indexes, establishing RRNG as the first provably robust foundation for RFANN indexing.

Similar to RNG, constructing the exact RRNG remains computationally expensive for large-scale datasets. To bridge theory and practice, we propose the Range-aware Neighborhood Search Graph (RNSG), a practical graph index that efficiently approximates the RRNG. The construction of RNSG proceeds in two stages: (1) building a *local RRNG* for each node by considering both spatial and attribute proximity to form its candidate neighbor set, and (2) merging these local structures to approximate the global RRNG. We develop an efficient algorithm for this construction and prove that the resulting RNSG meets two crucial properties of strong connectivity and structural heredity, ensuring robust performance under any range constraint. Furthermore, we design a corresponding query processing algorithm enhanced by a novel entry node generation technique. In summary, our main contributions are as follows.

Novel Theoretical Results. We propose a novel RRNG theory for RFANN search. The RRNG not only preserves the monotonic searchability of traditional RNG but also achieves a new property of structural heredity–guaranteeing that any range-induced subgraph remains a valid RRNG, thereby overcoming a fundamental limitation of previous methods. We further prove that RRNG achieves this with only a logarithmic overhead in both index size and search complexity compared to RNG. To our knowledge, RRNG is the first graph index that meets both monotonic searchability and structural heredity, establishing a theoretical foundation for future RFANN research.

New Practical Algorithms. Building upon the RRNG theory, we develop RNSG, a practical solution that efficiently approximates the RRNG. We present a fast range-aware pruning technique to accelerate RNSG construction, achieving a time complexity of $O(|C|M_{rnsg})$, where the parameters $|C|$ is a small constant in practice and M_{rnsg} denotes the number of edges of RNSG, ensuring scalability for large datasets. We prove that RNSG satisfies the strong connectivity and structural heredity, guaranteeing robust performance under any query range. Leveraging this index, we also devise an efficient query processing algorithm equipped with a carefully-designed entry node generation technique.

Extensive Experiments. We conduct extensive experiments on five real-world datasets, evaluating our method against 8 competitive baselines. The results demonstrate that: (1) RNSG significantly outperforms recent state-of-the-art methods including iRangeGraph [50] and UNIFY [30] in query performance. For instance, on the SIFT1M dataset, RNSG achieves 5645 QPS at 0.95 recall—representing a 2× and 4× improvement over iRangeGraph (2805 QPS) and UNIFY (1401 QPS) respectively; (2) RNSG maintains substantially more compact indexes, requiring several times less storage than competitors. For example, on the WIT dataset, RNSG maintains an index size of 0.38 GB, significantly smaller than iRangeGraph (1.5 GB) and UNIFY (8.9 GB). This represents a 3.9× to 23.4× reduction in memory overhead, demonstrating RNSG's exceptional space efficiency; (3) The index construction time of RNSG is substantially lower than iRangeGraph and UNIFY. For instance, on the WIT dataset, RNSG completes index construction in 1349 seconds, while iRangeGraph and UNIFY require 3495 and 11933 seconds respectively, representing a 2.59× to 8.85× speedup. Additional scalability tests confirm that RNSG maintains linear scaling with dataset size, underscoring its practical viability for large-scale applications. For reproducibility purposes, the source code of this paper is available at https://anonymous.4open.science/r/R-NSG-6BE1/.

2 Preliminaries

Let D be a dataset comprising n objects. Each object $o = (v, a)$ is composed of two features: a d-dimensional vector $o.v$, with d typically being hundreds, and a numerical attribute $o.a \in \mathbb{R}$.

Range-Filtered ANN Query (RFANN): Given a dataset D, an RFANN query is parameterized by a tuple $q = \langle v, I, k \rangle$, where $q.v$ is a query vector, $q.I = [a_l, a_r]$ is a numeric range, and $q.k$ is the number of desired results. The query returns $q.k$ objects from D that lie within the range $q.I$ and have the smallest Euclidean distance $\delta(q.v, o.v)$ to the query vector.

Following prior work [50], we assume without loss of generality that the dataset is sorted in ascending order by the numeric attribute[1], i.e., $o_i.a \le o_j.a$ for any $i \le j$. Given an RFANN query with range $[a_l, a_r]$, we can first use binary search to identify the smallest index L and the largest index R such that the subsequence $o_L, \ldots, o_R$ contains precisely all objects satisfying $a_l \le o_i.a \le a_r$. The RFANN query then simplifies to finding the k nearest neighbors to $q.v$ within this contiguous in-range subsequence.

Graph-based Indexes for ANNS. While a linear scan can find the exact k nearest neighbors for a query, it incurs substantial computational cost from calculating distances across the entire dataset, particularly for high-dimensional vectors. Thus, current state-of-the-art methods rely on graph-based indexes to efficiently find the approximate k nearest neighbors.

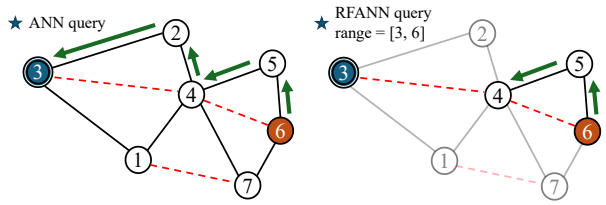
These methods construct a proximity graph $G = (V, E)$ during the indexing phase, where each node $x \in V$ corresponds to a data object $o \in D$. At query time, the beam search algorithm is typically employed [46]. Specifically, the beam search starts from an entry node (often the graph's approximate center) and maintains two dynamic sets: a candidate set $\mathcal{S}$ (a min-heap) and a result set $\mathcal{R}$ (a max-heap of size k). $\mathcal{R}$ stores the current k nearest neighbors, while $\mathcal{S}$ contains promising candidates for further exploration.

In each iteration, the algorithm expands the candidate with the smallest distance in $\mathcal{S}$. Its unvisited neighbors are then evaluated and added to $\mathcal{S}$. A neighbor is inserted into $\mathcal{R}$ if it is closer to the query than the farthest object currently in $\mathcal{R}$. To balance accuracy and efficiency, a hyperparameter ef_{search} limits the size of $\mathcal{S}$; when exceeded, the farthest candidate is removed. The search terminates when the minimum distance in $\mathcal{S}$ exceeds the maximum distance in $\mathcal{R}$, at which point $\mathcal{R}$ is returned as the final result.

Most state-of-the-art graph-based methods, including HNSW [33] and NSG [16], are built upon the concept of the Relative Neighborhood Graph (RNG) [43], which we introduce below.

Definition 2.1 (Relative Neighborhood Graph [43]). Given a dataset D in a metric space with distance function δ, the RNG is a graph $G(V, E)$ constructed on D such that, for any two nodes

[1]For expository clarity, we assume distinct attribute values; duplicates are resolved by enforcing a deterministic ordering based on object IDs, which does not affect the correctness and performance of our method.



(a) RNG for ANN query **(b) RNG for RFANN query**

Figure 1: Illustration of RNG for ANN and RFANN queries (red dashed edges are pruned by RNG). (a) Under a standard ANN query, the RNG correctly preserves a path to the true nearest neighbor (node 3). (b) Under an RFANN query with a range constraint, the required path is broken, making the true nearest neighbor unreachable from the query node.

$x, y \in V$, an edge $(x, y) \in E$ exists if and only if there is no third node $z \in V$ satisfying $\delta(x, z) < \delta(x, y)$ and $\delta(y, z) < \delta(x, y)$.

By Defintion 2.1, the longest edge of any triangle in RNG will be pruned. Fig. 1a illustrates the edge pruning strategy of the RNG, with red dashed lines representing the pruned edges. The RNG built upon this strategy[2] possesses significant theoretical advantages. As detailed below, it provides formal guarantees on both search result quality and time complexity [16].

LEMMA 2.2 ([16]). *Given a dataset D of n points in a metric space with distance function δ, let $G(V, E)$ be an RNG constructed on D. For any node $x \in V$, its nearest neighbor $y \in V$ can be found by performing a beam search on G.*

LEMMA 2.3 ([16]). *The time complexity of performing the beam search on the RNG is $O(n^{1/d} \log n) \approx O(\log n)$.*

Note that in Lemma 2.3, $n^{1/d}$ is a very small constant for high-dimensional vector data (e.g., if $d \geq 64$, $n^{1/d} \leq 2$ even when $n = 2^{64}$). Fig. 1a illustrates this theoretical advantage: a greedy beam search starting at node 6 correctly converges to node 3, the true nearest neighbor of the query. The performance benefits of this pruning strategy were empirically confirmed in [47].

However, constructing an exact RNG for a dataset of size n has a time complexity of $O(n^3)$ [16], which is prohibitively expensive. Consequently, practical methods (e.g., HNSW and NSG) construct graph indexes by approximating the RNG [43]. Typically, they first retrieve hundreds of candidate neighbors (e.g., via approximate nearest neighbor search) and then apply the RNG pruning rule to this candidate set. A hyperparameter m limits the maximum out-degree of each node, ensuring sparsity by retaining only the top-m neighbors after pruning [47].

Existing RFANN Search Methods. Various methods have been proposed for RFANN queries [45], which can be categorized into three basic strategies based on when the range filter is applied: *pre-filtering*, *post-filtering*, and *in-filtering*. (1) *Pre-filtering* first applies the range filter to obtain a set of in-range objects, then performs a nearest neighbor search over this subset. However, it suffers from a linear scan cost for unselective ranges with large result sets. (2) *Post-filtering* first executes a standard ANN search and then filters the results by the range condition. This strategy, conversely, incurs significant redundant computation for highly selective queries, as many searched neighbors may be out-of-range. (3) *In-filtering* integrates the range check directly into the ANN search process, evaluating only in-range objects. While this approach is intuitive, it can compromise the proximity properties of the underlying index (e.g., the RNG property in graph-based indexes), often leading to suboptimal performance.

To handle diverse query workloads, several hybrid strategies have been proposed [13, 30, 45]. For instance, Milvus [45] employs a cost-based model to dynamically select the optimal strategy (pre-, post-, or in-filtering) for each query. SuperPostfiltering [13] predefines multiple overlapping ranges and builds a dedicated graph index for each. At query time, it selects the smallest pre-built range covering the query interval and performs a post-filtering search within it. UNIFY [30] partitions the data by numeric attributes, building separate proximity graphs for each segment. It uses skip lists for fast segment location and edge bitmaps for range-based pruning, finally applying a cost model to select the best search strategy within segments. Despite their sophistication, these methods are fundamentally constrained by the inherent limitations of the basic strategies, leading to sub-optimal performance and high indexing costs, as shown in our experiments (see Section 5).

To overcome these limitations, recent studies [50, 54] have integrated numeric attributes directly into graph-based ANN indexes. For instance, [54] proposes to construct a dedicated graph for every possible query range. While theoretically optimal, this approach is impractical as it requires building $O(n^2)$ indexes. To mitigate this cost, [54] employs aggressive compression that discards a substantial number of edges. Paradoxically, this undermines the method's effectiveness, as [50] shows that it fails to achieve a recall above 0.8 for short-range queries. Seeking a better trade-off, [50] introduces the *iRangeGraph*, which integrates a segment tree with an ANN index. It constructs elemental graphs for a moderate number of ranges and dynamically composes them at query time. Although this improves query performance, the storage of redundant information across elemental graphs incurs massive indexing overhead (as confirmed in our experiments).

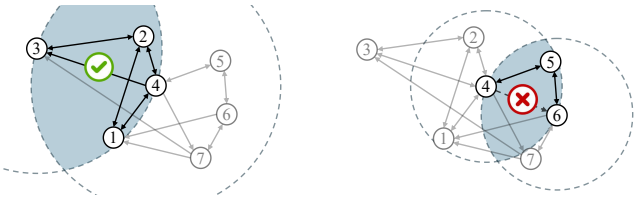
Limitations of Existing RFANN Methods. Existing RFANN methods predominantly attempt to directly combine graph-based ANN indexes with numeric attributes. This strategy is fundamentally challenging because range filters can disrupt the distance-based proximity properties (e.g., the RNG) essential for efficient graph traversal. As shown in Fig. 1b, a range constraint can break critical graph edges, making the true nearest neighbor unreachable during search.

This leads to a clear dilemma: either build expensive multi-range indexes for performance, or accept degraded performance for lower cost. *iRangeGraph* [50] exemplifies the first approach, achieving top-tier recall at the cost of indexing overhead an order of magnitude greater than other approaches. Conversely, cheaper methods are often over an order of magnitude slower at high recall (e.g., ≥0.95). In this work, we address this dilemma by introducing a novel graph index that simultaneously achieves high query performance and low indexing overhead.

3 Range-aware Relative Neighborhood Graph

To overcome the limitations of existing RFANN methods, we establish a novel graph indexing theory: the Range-aware Relative Neighborhood Graph (RRNG), which serves as the theoretical foundation for our RNSG method (detailed in Section 4). The RRNG retains the desirable searchability of the RNG, guaranteeing that beam search will identify the nearest neighbor from any node $x \in V$

[2] [16] refers to the graph constructed based on the RNG's pruning strategy as the Monotonic Relative Neighborhood Graph (MRNG). For simplicity, we use the term RNG throughout this paper.



(a) Edges retained by RRNG (b) Edges pruned by RRNG

Figure 2: Illustration of the RRNG pruning strategy

in the graph $G(V, E)$. More importantly, we prove that the RRNG satisfies a crucial *hereditary property*: for any query range $q.I$, the induced subgraph $G[I]$, consisting of in-range nodes and edges, is equivalent to the RRNG constructed on the in-range objects. This ensures both the correctness and efficiency of RFANN query processing. We begin by formally defining the RRNG, followed by an analysis of its theoretical properties.

Definition 3.1 (RRNG). Given a dataset D in a metric space with distance function δ, the Range-aware Relative Neighborhood Graph (RRNG) is a graph $G(V, E)$ constructed on D. For any two nodes $x, y \in V$, we assume without loss of generality that $x.a < y.a$. The edge $(x, y) \in E$ exists if and only if there is no another edge $(x, z) \in E$ satisfying the following conditions:

$$\delta(x, z) < \delta(x, y), \quad \delta(y, z) < \delta(x, y), \quad \text{and} \quad x.a < z.a < y.a.$$

By Definition 3.1, the longest edge in a triangle is pruned in an RRNG if the attribute of the third node lies within the attribute range defined by the two endpoints of that edge.

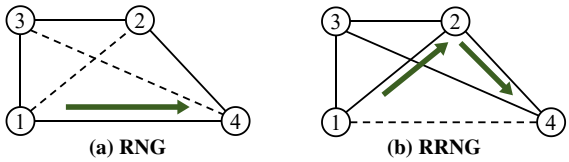
Example 1. *Fig. 2 illustrates the RRNG structure, with node numbers indicating numeric attributes. Subfigure (a) depicts a difference from the RNG: while the RNG will prune edge* $(3, 4)$ *as the longest edge in triangle* $(2, 3, 4)$*, our RRNG retains it because node* 2 *falls outside the attribute range of the edge endpoints. Subfigure (b) shows the RRNG's pruning strategy: edge* $(4, 6)$ *is pruned as it is the longest edge in triangle* $(4, 5, 6)$*, and node* 5 *lies within the attribute range of* $(4, 6)$.

Differences between RNG and RRNG. Although Example 1 demonstrates the fundamental differences between RRNG and RNG, one might intuitively assume that the RNG is a subgraph of RRNG, since RRNG's range-aware pruning condition appears stricter and thus might preserve more edges. However, this is not true. As shown in Fig. 3, the RNG in subfigure (a) prunes edges $(1, 2)$ and $(3, 4)$, while the corresponding RRNG in subfigure (b) retains these edges but prunes edge $(1, 4)$—which exists in the RNG—because node 2 lies within the attribute range defined by nodes 1 and 4, and edge $(1, 4)$ is the longest in triangle $(1, 2, 4)$. This confirms that the edge sets of RNG and RRNG are not in a subset relationship.

In summary, RNG establishes edges based solely on *spatial* proximity, whereas RRNG incorporates both *spatial* and *attribute* proximity. This design principle leads to distinct graph topologies: direct edges connecting distant attributes (e.g., $(1, 4)$ in Fig. 3a) in RNG are often replaced in RRNG by paths traversing nodes with small attribute differences (e.g., $(1, 2)$ and $(2, 4)$ in Fig. 3b).

3.1 Theoretical Properties of RRNG

In this subsection, we establish two crucial theoretical properties of our RRNG: monotonic searchability property and hereditary property. We begin by defining the concept of a monotonic path.



(a) RNG (b) RRNG

Figure 3: Differences between RNG and RRNG. The RRNG tends to replace edges (e.g., the edge $(1, 4)$) with large attribute differences with those having smaller attribute differences (e.g., the edges $(1, 2)$ and $(2, 4)$). In the RNG, $1 \to 4$ is a monotonic path, while in the RRNG $1 \to 2 \to 4$ is a monotonic path.

Definition 3.2 (Monotonic Path). Given a graph $G(V, E)$ and a distance function δ, a directed path from node $x \in V$ to $y \in V$, denoted by $x = v_0 \to v_1 \to \cdots \to v_m = y$, is referred to as a monotonic path if and only if $\delta(v_{i+1}, y) < \delta(v_i, y), \quad \forall i \in \{0, 1, \ldots, m-1\}$.

The following theorem shows that for any two nodes x and y in RRNG, there exists a monotone path from x to y.

Theorem 3.3 (Monotonic Searchability Property). *Given a dataset D in a metric space with distance function δ, let $G = (V, E)$ be an RRNG constructed over D as defined in Definition 3.1. Then, for any two distinct nodes $x, y \in V$, there exists a monotonic path from x to y in G, along which the distance to y strictly decreases at every step.*

Proof. Without loss of generality, we fix a target node $q \in V$ and consider all other nodes $x \in V \setminus \{q\}$, ordered by increasing distance to q:

$$\delta(x_1, q) < \delta(x_2, q) < \cdots < \delta(x_n, q).$$

By induction on this order, we show that every node x_i has a strictly distance-decreasing path to q.

Base step: By the pruning strategy of the RRNG, the edge $(x_1, q) \in E$ necessarily exists, as it corresponds to the smallest distance. Hence, the path $x_1 \to q$ is strictly decreasing.

Inductive step: Assume that every x_j with $\delta(x_j, q) < \delta(x_i, q)$ already has a strictly distance-decreasing path to q. Then, we have two cases:

Case 1: If $(x_i, q) \in E$, that edge itself forms such a path.

Case 2: Otherwise, it is pruned by the RRNG's pruning strategy. Then, there must exist $r \in V$ with

$$\delta(r, q) < \delta(x_i, q), \quad \delta(x_i, r) < \delta(x_i, q), \quad (x_i, r) \in E.$$

By the inductive hypothesis, node r has a strictly distance-decreasing path $r \to \cdots \to q$, and every node x_j on this path satisfies $\delta(x_j, q) < \delta(x_i, q)$. Then, it is easy to show that concatenating the edge (x_i, r) with this path yields a strictly distance-decreasing path from x_i to q. This completes the proof. □

Note that the RNG also satisfies the monotonic searchability property [16]. Fig. 3a illustrates a monotonic path ($1 \to 4$) in RNG, while Fig. 3b depicts a monotonic path ($1 \to 2 \to 4$) in RRNG. The monotonic searchability property guarantees that for any data vector $y.v \in D$, a beam search algorithm will correctly identify the nearest neighbor $y \in V$ in the RRNG $G = (V, E)$.

Corollary 3.4. *Let $G = (V, E)$ be an RRNG constructed from a dataset D in a metric space with distance function δ. For any vector $y.v \in D$, its nearest neighbor can be found using the beam search algorithm in Section 2, regardless of the entry node x.*

Proof. Lemma 3.3 guarantees the existence of a monotonic path from any node x to any target y: at each step, there is a neighbor closer to y. Now consider the beam search process: the candidate min-heap $\mathcal{S}$ maintains the set of visited nodes prioritized by their distance to y. Initially, $\mathcal{S}$ contains the start node. In each iteration, the algorithm expands the node in $\mathcal{S}$ closest to y. Due to the monotonicity property, among its neighbors, there must exist at least one node that is closer to y. This node will be added to $\mathcal{S}$ and, due to its smaller distance, will eventually become the top of the heap. Therefore, the minimum distance in $\mathcal{S}$ strictly decreases with each step until y itself is reached and returned. □

In addition to the monotonic searchability, the RRNG satisfies a crucial *hereditary* property that enables efficient processing of RFANN queries.

Theorem 3.5 (Hereditary Property). *Let $G = (V, E)$ be an RRNG constructed from a dataset D in a metric space with distance function δ. For any RFANN query $q = \langle v, I = [a_l, a_r], k\rangle$, the induced subgraph $G[I] = (V', E')$, where $V' = \{x \in V | x.a \in [a_l, a_r]\}$ and $E' = \{(u, v) | (u, v) \in E, u, v \in V'\}$, is equivalent to an RRNG constructed on V'.*

Proof. For any query interval $I = [a_l, a_r]$, let $G = (V, E)$ be the RRNG on the full dataset, and $G[I] = (V_I, E')$ be the subgraph induced by nodes in I, where V_I denotes the node subset filtered by the range I. Let $H[I] = (V_I, E_I)$ be the RRNG constructed directly on V_I. Since V_I is common to both, we prove $E' = E_I$ by induction on all pairs $(x, y) \in V_I \times V_I$ sorted by $\delta(x, y)$, assuming without loss of generality $x.a < y.a$.

Base Case: Consider the pair (x, y) with minimal $\delta(x, y)$. For edge absence, any witness z must satisfy $x.a < z.a < y.a$ and thus $z \in V_I$. However, since $\delta(x, y)$ is minimal, no such z can make (x, y) the longest edge in the triangle (x, y, z). Therefore, (x, y) must exist in both E' and E_I.

Inductive Step: Assume all pairs with distance smaller than $\delta(x, y)$ have identical edge status in E' and E_I.

Case 1: If $(x, y) \notin E'$, then $\exists z \in V$ with $(x, z) \in E'$ and $x.a < z.a < y.a$, where (x, y) is the longest edge in the triangle (x, y, z). Since $z \in V_I$ and $\delta(x, z), \delta(y, z) < \delta(x, y)$, by the induction hypothesis, $(x, z) \in E_I$. Thus, (x, y) is correctly absent in E_I.

Case 2: If $(x, y) \in E'$, then no such z exists in V. Since $V_I \subseteq V$, no such z exists in V_I either. Therefore, $(x, y) \in E_I$.

By induction, $E' = E_I$, proving $G[I] = H[I]$. □

By Theorem 3.5, the subgraph induced from the RRNG by any query range remains a *correct* RRNG, preserving all its theoretical properties. Consequently, for any RFANN query, the induced subgraph corresponding to the query range can serve directly as an effective search index without performance degradation. This eliminates the need to precompute and store multiple range-specific graph indexes—a major limitation of previous methods [13, 50, 54]—thus dramatically reducing indexing overheads.

Example 2. *Fig. 4 illustrates the hereditary property described in Theorem 3.5. Given an RFANN query with $q.r = [3, 6]$, the induced subgraph preserves the search property of the RRNG, ensuring that the nearest neighbor of any node can be reached using beam search.*

3.2 Complexity Analysis

This subsection presents complexity analyses of the RRNG. We begin by examining the time complexity of beam search on the RRNG, then proceed to analyze the RRNG size, and finally establish the time complexity of its construction. Due to space limitations, all missing proofs are provided in the full version [4].

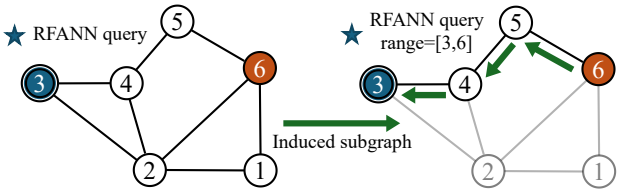


Figure 4: Illustration of the hereditary property of RRNG. The left subfigure shows the complete RRNG graph, while the right subfigure depicts the subgraph induced by a query range $[3, 6]$.

Theorem 3.6. *The time complexity of performing beam search on an RNNG is $O(n^{1/d} \log^2 n) \approx O(\log^2 n)$.*

Theorem 3.6 shows that the RRNG supports RFANN queries over arbitrary query ranges while incurring only a modest logarithmic factor, $\log(n)$, in time complexity compared to standard RNG for ANN search.

Theorem 3.7. *The size of an RRNG is bounded by $O(S_r \log n)$, where S_r is the expected size of RNG.*

Theorem 3.7 shows the RRNG space overhead is $O(\log n)$ relative to the RNG. However, by storing just a single graph index instead of many, our method is much more space-efficient than most existing RFANN algorithms that precompute multiple graph indexes for different ranges [13, 50, 54].

Basic Approach to Construct RRNG. Definition 3.1 suggests a basic algorithm for constructing the exact RRNG. The key insight is that the pruning of the longest edge in any triangle (x, y, z) depends on the existence of the two shorter edges. Therefore, we process all node pairs (x, y) in ascending order of their distance $\delta(x, y)$. For each pair, we check all other nodes z as potential witnesses. The edge (x, y) is added to the graph if and only if no witness z satisfies the RRNG pruning condition for the triangle (x, y, z). This ordering guarantees that the existence of the shorter edges (x, z) and (y, z) has already been determined when processing (x, y), ensuring the correctness of the algorithm. The time complexity of this basic approach is presented in the following theorem.

Theorem 3.8. *The time complexity of the basic RRNG construction algorithm is $O(n^3)$.*

Proof. The complexity of the basic RRNG construction algorithm arises from two steps: first, sorting all node pairs by their vector distance, which takes $O(n^2 d)$ time; second, verifying the pruning condition for each pair of nodes by enumerating all possible third nodes, which requires $O(n^3)$ time. Given that $d < n$ in practice, the overall complexity is $O(n^3)$. □

By Theorem 3.8, constructing an RRNG graph requires $O(n^3)$ time, the same as that of the RNG [2], which makes it impractical. Therefore, in the next section, we propose a novel approach to construct an approximate RRNG as a graph index, referred to as the range-aware neighborhood search graph (RNSG).

4 The Proposed RNSG Approach

Given the computational expense of exact RRNG construction, we propose the range-aware neighborhood search graph (RNSG), a practical approach that efficiently approximates the RRNG. The key idea

Algorithm 1: Fast Range-Aware Pruning Strategy

```
   Input: A node x, a candidate set C for x, maximum edges m
   Output: Neighbor set NS of x
1  Procedure RRNGPrune(x, C, m)
2      NS ← ∅;
3      C_l ← { v ∈ C | v.a < x.a };
4      C_r ← { v ∈ C | v.a > x.a };
5      Sort C_l and C_r in ascending order of |v.a − x.a|;
6      R_l ← Prune(x, C_l, m/2);
7      R_r ← Prune(x, C_r, m/2);
8      NS ← R_l ∪ R_r;
9      return NS ;
10 Procedure Prune(x, C, m)
11     R ← ∅;
12     for v_i ∈ C do
13         keep ← True;
14         for v_j ∈ R do
15             if δ(x, v_j) < δ(x, v_i) and
16                 δ(v_i, v_j) < δ(x, v_i) then
17                 keep ← False;
18                 break;
19         if keep and |R| < m then
20             R ← R ∪ v_i;
21     return R;
```

of RNSG is to approximate the global RRNG by constructing and merging *local RRNGs* for each node. This is achieved in two stages: first, for each node, we identify a candidate set of neighbors that considers both spatial and attribute proximity; second, we apply the RRNG pruning rule (Definition 3.1) to this candidate set to build the local graph. In the following subsections, we first propose a novel range-aware pruning strategy, then detail our RNSG construction algorithm, and finally present the RFANN query processing algorithm with RNSG.

4.1 Fast Range-Aware Pruning Strategy

We now introduce a novel and efficient range-aware pruning strategy, which forms the core of RNSG and is detailed in Algorithm 1. The algorithm initializes an empty neighbor set NS (line 2). A key step is to partition the candidate set C into two subsets, C_l and C_r, containing objects with attribute values less than and greater than $x.a$, respectively (lines 3–4). This division, justified by Lemma 4.1, ensures that pruning can be performed independently within each subset, thus significantly improving efficiency.

LEMMA 4.1. *For any node $x \in C$, we partition the candidate set C into two sets C_l and C_r where $C_l = \{y \in C | y.a < x.a\}$ and $C_r = \{y \in C | y.a > x.a\}$. Then, this partition ensures independence in RRNG pruning: edges from x to any node in C_l are unaffected by nodes in C_r, and vice versa.*

PROOF. For any $v_l \in C_l$, we have $v_l.a < x.a$, and for any $v_r \in C_r$, we have $v_r.a > x.a$. Taking any pair of nodes v_l and v_r from C_l and C_r respectively, $v_l.a < x.a < v_r.a$ must hold. Thus, neither of our pruning conditions $v_l.a < v_r.a < x.a$ nor $x.a < v_l.a < v_r.a$ holds. Therefore, for any $v_l \in C_l$, v_r cannot affect its pruning. Similarly, for any $v_r \in C_r$, v_l cannot affect its pruning. □

For a node x, after partitioning the candidate set C into C_l and C_r, we sort each subset by the absolute difference between the candidate's attribute and $x.a$ (line 5). This ordering is crucial because Lemma 4.2 guarantees that a candidate with a smaller attribute gap cannot be pruned by another candidate with a larger gap.

LEMMA 4.2. *Let x be a node and $C_l = \{v_i \in C | \ v_i.a < x.a\}$ be sorted by $|x.a - v_i.a|$ in ascending order. For any $c_i, c_j \in C_l$ with $|c_i.a - x.a| < |c_j.a - x.a|$, the edge (x, c_i) cannot be pruned by (x, c_j). The case for $C_r = \{v_i \in C | \ v_i.a > x.a\}$ is symmetric.*

PROOF. When $v_i \in C_l$, any node satisfying $|c_i.a - x.a| < |c_j.a - x.a|$ must have $c_i.a > c_j.a$. Therefore, the pruning condition of edge (x, c_i), $c_i.a < c_j.a < x.a$ is not satisfied, thus preventing the pruning of (x, c_i). Similarly, consider the case where $c_i \in C_r$ and $c_i.a < c_j.a$. Then, the pruning condition $x.a < c_j.a < c_i.a$ is again violated. Thus, the lemma holds for both scenarios. □

Lemma 4.2 enables us to process nodes in ascending order of their attribute gap to x (line 5). This ordered processing ensures no valid edges are missed. We then perform sequential pruning within each subset, initializing a result set R and inserting nodes in the sorted order (line 12). A node is pruned if it forms the longest edge with any previously added neighbor (lines 13-20). Finally, we select at most $m/2$ edges from each subset (lines 6-7), merge them (line 8), and return the final neighbor set (line 9). Here, m is a hyper-parameter used to control the size of the neighborhood. Note that if we set the candidate set C to the entire dataset D and set $m = \infty$, then we can use Algorithm 1 to construct the exact RRNG, indicating the correctness of our range-aware pruning strategy.

THEOREM 4.3. *For a dataset D, if Algorithm 1 is applied to every object $x \in D$ with parameters $C = D$ and $m = \infty$, the resulting graph, comprising all objects and their computed neighborhood sets $NS(x)$, precisely yields the RRNG defined in Definition 3.*

Below, we show that the time complexity of RRNG construction can be reduced from $O(n^3)$ to $O(nM_{rrng})$ ($M_{rrng} < n^2$) by employing our fast range-aware pruning strategy, where M_{rrng} denotes the number of edges in RRNG.

THEOREM 4.4. *The time complexity of constructing an exact RRNG using Algorithm 1 is $O(nM_{rrng})$.*

PROOF. Let $deg(v)$ be the out-degree of node v in the RRNG $G(V, E)$. The construction involves executing Algorithm 1 for each node $v \in V$ with parameters $C = V$ and $m = \infty$. For a single node v, the algorithm runs in $O(n \cdot |R|)$ time, which is bounded by $O(n \cdot deg(v))$. Summing over all nodes, and noting that $\sum_{v \in V} deg(v) = M_{rrng}$, the overall complexity is $O(nM_{rrng})$. □

Although our fast range-aware pruning strategy reduces the time complexity of RRNG construction to $O(nM_{rrng})$, this cost remains prohibitive for large-scale applications. To further improve the efficiency, we present RNSG, a practical graph index that leverages our range-aware pruning strategy to efficiently approximate the RRNG.

4.2 The RNSG Construction Algorithm

Algorithm 2 details the construction of the RNSG index. The process begins by building a neighborhood for each node x that captures both spatial and attribute proximity. Spatially, we employ the classic NNDescent algorithm [11] to construct an approximate k-nearest neighbor graph (KNNG) (line 2), which identifies neighbors based

on the distance function δ. For attribute proximity, we consider any node x whose attribute value lies within a threshold $ef_{\text{attribute}}$ ($ef_{\text{attribute}} \geq 1$) of $v.a$ as an *attribute-similar* neighbor (line 3, lines 7-8). The final candidate neighbor set for each node v is formed by merging its *spatial neighbors* from the approximate KNNG with $degree(v) = ef_{\text{spatial}}$and its *attribute-similar neighbors* (lines 3-8).

Subsequently, we apply Algorithm 1 to prune this candidate set, obtaining the final neighbor list for each node (line 9). Finally, the RNSG is obtained from the union of all edges (lines 10-12). The time complexity of Algorithm 2 is presented in the following theorem.

THEOREM 4.5. *The time complexity of Algorithm 2 is $O(|C|M_{rnsg})$, where M_{rnsg} is the number of edges in the final RNSG and $|C|$ denotes the maximum neighborhood size.*

PROOF. Similar to the proof of Theorem 4.4, we apply Algorithm 1 on all the n nodes with candidate size $|C|$, leads to each $O(\sum deg_{RNSG}(v)|C|)$ time complexity, which can be simplify as $O(|C|M_{rnsg})$. □

In Theorem 4.5, $|C|$ is clearly bounded by $2ef_{\text{attribute}} + ef_{\text{spatial}}$ (where ef_{spatial} is from NNDescent [11]. Since $|C| \ll n$ in practice, our method is highly efficient and scalable.

Nice Properties of RNSG. We first show that the RNSG graph constucted by Algorithm 2 is strongly connected. Moreover, for any query range $q.r$, the induced subgraph $G[r]$ of the RNSG is also strongly connected.

THEOREM 4.6 (STRONG CONNECTIVITY). *The RNSG graph G and any range-induced subgraph $G[I]$ are both strongly connected.*

PROOF. In Algorithm 2, given the node set $V = \{v_1, v_2, \cdots, v_n\}$ sorted in ascending order of $v.a$ (the numeric attribute), for any node v_i ($i + 1 \leq n$), v_{i+1} must be included in its candidate set. This follows because no other node's attribute lies strictly between $v_i.a$ and $v_{i+1}.a$. Consequently, v_{i+1} must be present in the final neighborhood set of v_i after pruning, and by symmetry, v_i is also in the neighborhood of v_{i+1}. This pairwise connectivity forms a bidirectional chain along the sorted order, guaranteeing that both the RNSG and any interval-induced subgraph are strongly connected. □

The strong connectivity in Theorem 4.6 guarantees that beam search with a sufficiently large beam size can find the nearest neighbor of any node v in the RNSG G for any query range. While the RNSG, as an approximation of the RRNG, cannot preserve the monotonic search property and its theoretical search performance may be inferior to the exact RRNG, our experimental results demonstrate that it achieves very high search performance in practice. Similar to RRNG, we show that our RNSG also satisfies the hereditary property.

THEOREM 4.7 (HEREDITARY PROPERTY). *Let $G = (V, E)$ be an RNSG constructed from a dataset D in a metric space with distance function δ and a KNNG G'. For any RFANN query $q = \langle v, I = [a_l, a_r], k\rangle$, the induced subgraph $G[I] = (V', E')$, where $V' = \{x \in V | x.a \in [a_l, a_r]\}$ and $E' = \{(u, v)|(u, v) \in E, u, v \in V'\}$, is equivalent to an RNSG constructed on V' with the induced graph of KNNG $G'[I]$.*

The hereditary property of the RNSG, established in Theorem 4.7, enables a single graph index to support arbitrary RFANN queries, thereby ensuring significantly low indexing costs compared to methods requiring multiple range-specific indexes [13, 50, 54].

Algorithm 2: The RNSG Construction Algorithm

Input: Dataset D, attribute threshold $ef_{\text{attribute}} \geq 1$, and maximum edges m
Output: A RNSG $G = (V, E)$
1 **Function** `RNSGConstruct` $(D, r, ef_{\text{attribute}})$
2 Obtain an approximate KNN graph $G' = \{V', E'\}$ from D;
3 Let $\{v_1, \cdots, v_n\}$ be sorted in ascending order by the nodes' numeric attributes;
4 Initialize $G \leftarrow (\{v_1, \cdots, v_n\}, E')$;
5 **for** $i \leftarrow 1$ **to** n **do**
6 $C \leftarrow$ Neighbour(v_i);
7 **for** $j \in [i - ef_{\text{attribute}}, i + ef_{\text{attribute}}]$ **do**
8 C.add(v_j);
9 Neighbour$(v_i) \leftarrow$ `RRNGPrune` (v_i, C, m);
10 **foreach** $x \in$ Neighbour(v_i) **do**
11 $E \leftarrow E \cup \{(v_i, x)\}$;
12 **return** G;

4.3 Query Processing Algorithm With RNSG

We now present the RFANN query processing algorithm using the RNSG index. Similar to other graph-based methods [5, 47], we employ a beam search. Crucially, for any range query I, our method does not explicitly materialize the induced subgraph. Instead, the search is performed on the entire RNSG, with the range filter applied on-the-fly during traversal to prune edges that lead to out-of-range nodes. It is important to emphasize that our algorithm differs fundamentally from conventional *in-filter* methods. Traditional in-filter approaches often suffer from degraded navigability and even graph disconnection, as range filters prune edges outside the query interval, drastically reducing the effective connectivity of the search graph. In contrast, our method is theoretically grounded by the hereditary property (Theorem 4.7), which guarantees that for any query range, the range-induced subgraph preserves the original RNSG's connectivity and navigability. This ensures that the range filtering process does not compromise the structural property essential for efficient search.

Entry Node Generation. As observed in previous studies [5, 46, 50], the starting nodes or the entry nodes of the beam search could affect the search performance. Traditional graph-based ANN methods typically use the *center node* as the entry node, which is the node closest to the centroid of the entire datasets [16, 46]. However, for RFANN queries, the center node varies with different range constraints. A naive approach would be to precompute and store the center node for every possible range; however, this requires $O(n^2)$ space, making it impractical.

To overcome this issue, we develop an effective heuristic for identifying entry nodes. Our approach is motivated by a mild assumption: the vector embedding distribution is independent of the attribute value distribution. This implies that the centroids of datasets filtered by different range constraints should remain relatively stable. It is important to note that this centroid is a point in the vector space and is not necessarily an actual data object in the dataset. Consequently, we can leverage the global centroid of the entire dataset as a high-quality approximation for the centroid of any range-filtered subset. The entry node for a given range is then selected as the in-range node closest to this global centroid. Building upon this heuristic, we prove a key theoretical result: for a fixed right endpoint

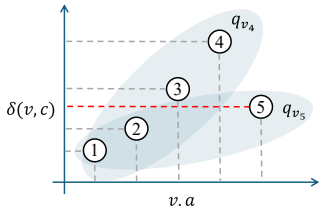


Figure 5: Illustration of entry node set generation

r, the expected number of distinct entry nodes required, as the left endpoint varies from 1 to r, is only $O(\log n)$. In other word, for all sub-ranges of $[1, r]$, the total number of distinct entry nodes is bounded by $O(\log n)$.

LEMMA 4.8. *Assume that the vector embedding distribution is independent of the attribute value distribution, the expected number of distinct entry nodes required for all query ranges with a fixed right endpoint a_r is $O(\log n)$.*

Based on Lemma 4.8, we present an efficient algorithm to precompute entry nodes, as outlined in Algorithm 3. The procedure begins by computing the global centroid c of the dataset (line 2) and initializing a result set T and a candidate set q (lines 3–4). After sorting all nodes by their attribute value (line 5), we process them in ascending order (line 6). For each node v_i, we compute its distance to c and remove from q any earlier node v_j ($j < i$) satisfying $\delta(v_j, c) > \delta(v_i, c)$ (line 7). This ensures that q always maintains a sequence of nodes with increasing attribute values and non-decreasing distances to c. Formally, for any $v_j, v_k \in q$ with $j < k$, we have $\delta(v_j, c) \leq \delta(v_k, c) \leq \delta(v_i, c)$. Intuitively, q contains candidate entry nodes for ranges ending at v_i's attribute, where the node with the smallest attribute in q is the one closest to c. The current q is stored in T for each v_i (line 9). By Lemma 4.8, each q has size $O(\log n)$, so the total size of T is $O(n \log n)$. At query time, for a range $[a_l, a_r]$, we locate the in-range node with attribute closest to a_r, retrieve its corresponding q from T, and select the node in q with the smallest attribute value as the entry node. This node is, by construction, the closest to the global centroid c among all nodes in the range $[a_l, a_r]$. The following example illustrates the operation of Algorithm 3.

EXAMPLE 3. *In Fig. 5, the horizontal axis denotes the attribute $v.a$ and the vertical axis is the distance $\delta(v, c)$ from each node to the global centroid c. At v_4, the entry set is $q_{v_4} = \{v_1, v_2, v_3, v_4\}$. When the algorithm processes v_5, its distance (red dashed line) satisfies $\delta(v_5, c) < \delta(v_3, c)$ and $\delta(v_5, c) < \delta(v_4, c)$, so the algorithm removes v_3 and v_4 and obtains $q_{v_5} = \{v_1, v_2, v_5\}$. For any query whose right endpoint is $v_5.a$, v_5 is a strictly better entry candidate than v_3 and v_4, hence both v_3 and v_4 are no longer needed. This matches the update rule in Algorithm 3 (lines 7-9): upon processing v_i, remove all $x \in q$ with $\delta(x, c) > \delta(v_i, c)$ and then insert v_i into q, keeping q monotone in $\delta(\cdot, c)$.*

5 Experiments

5.1 Experimental Setup

Datasets. Following previous RFANN studies [28, 30, 50, 54], we use 5 widely-used real datasets for evaluation: YouTube-Audio[3] (YT-Audio for short), YouTube-RGB[4] (YT-RGB for short), WIT-Image[5] (WIT for short), SIFT1M and GIST1M[6]. Dataset statistics are summarized in Table 1. For numeric attribute assignment, we maintain consistency with previous studies [28, 50]: YT-Audio uses video publication time, YT-RGB employs *like* counts, and WIT utilizes image size. For SIFT1M and GIST1M, we generate synthetic numeric attributes using random distributions following the methodology used in [28].

[3]https://research.google.com/youtube8m/download.html
[4]https://research.google.com/youtube8m/download.html
[5]https://github.com/google-research-datasets/wit

Algorithm 3: Entry Nodes Generation

Input: Node set V
Output: A List of entry node set T
1 **Function** `GenEntry(V)`
2 Calculate the centroid c of the node in V;
3 Initialize $T \leftarrow \emptyset$;
4 Initialize $q \leftarrow \emptyset$;
5 Sort V by nondecreasing attribute values to obtain $\langle v_1, v_2, \ldots, v_n \rangle$;
6 **for** $i \leftarrow 1$ **to** n **do**
7 Remove all node x in q such that $\delta(x, c) > \delta(v_i, c)$;
8 $q \rightarrow q \cup \{v_i\}$;
9 $T \rightarrow T \cup \{q\}$;
10 **return** T;

Table 1: Dataset Statistics

Datasets	$\lvert D \rvert$	#Query	d	Vector Type
YT-Audio	1M	1,000	128	audio
YT-RGB	1M	1,000	1,024	video
WIT	1M	1,000	2,048	image
SIFT1M	1M	10,000	128	image
GIST1M	1M	1,000	960	image

Query Ranges. Consistent with prior research [50], we note that despite the widespread industrial adoption of RFANN queries (e.g., at Apple [41], Milvus [45], and Alibaba [49]), no benchmark datasets and query ranges are publicly available due to privacy considerations. Following established evaluation methodologies [28, 50], we generate synthetic query ranges to assess performance. We define the range fraction of a query as 2^{-i} if it covers $n/2^i$ data objects, and categorize queries into three selectivity levels: large-range ($i \in [0, 3]$), moderate-range ($i \in [4, 6]$), and small-range ($i \in [7, 9]$). Our evaluation employs two strategies: (1) overall performance assessment using diverse query workloads, where the query set is evenly divided into ten subsets corresponding to range fractions 2^{-i} for $i = 0$ to 9; and (2) specific performance analysis under fixed selectivity levels, where we fix the selectivity of the entire query set at 1%, 10%, and 25% to systematically evaluate method performance across varying selectivity levels.

Evaluation Metrics. Following previous studies [46, 47], we use two standard metrics to assess method performance. Accuracy is measured by recall@$k = \frac{|\mathcal{R} \cap \tilde{\mathcal{R}}|}{k}$, where $\mathcal{R}$ represents the result set retrieved by the evaluated method and $\tilde{\mathcal{R}}$ denotes the ground-truth set obtained through exhaustive linear scan. Consistent with recent RFANN studies [28, 30, 50], we primarily report recall@10 to facilitate direct comparison with state-of-the-art methods, while also

[6]http://corpus-texmex.irisa.fr/

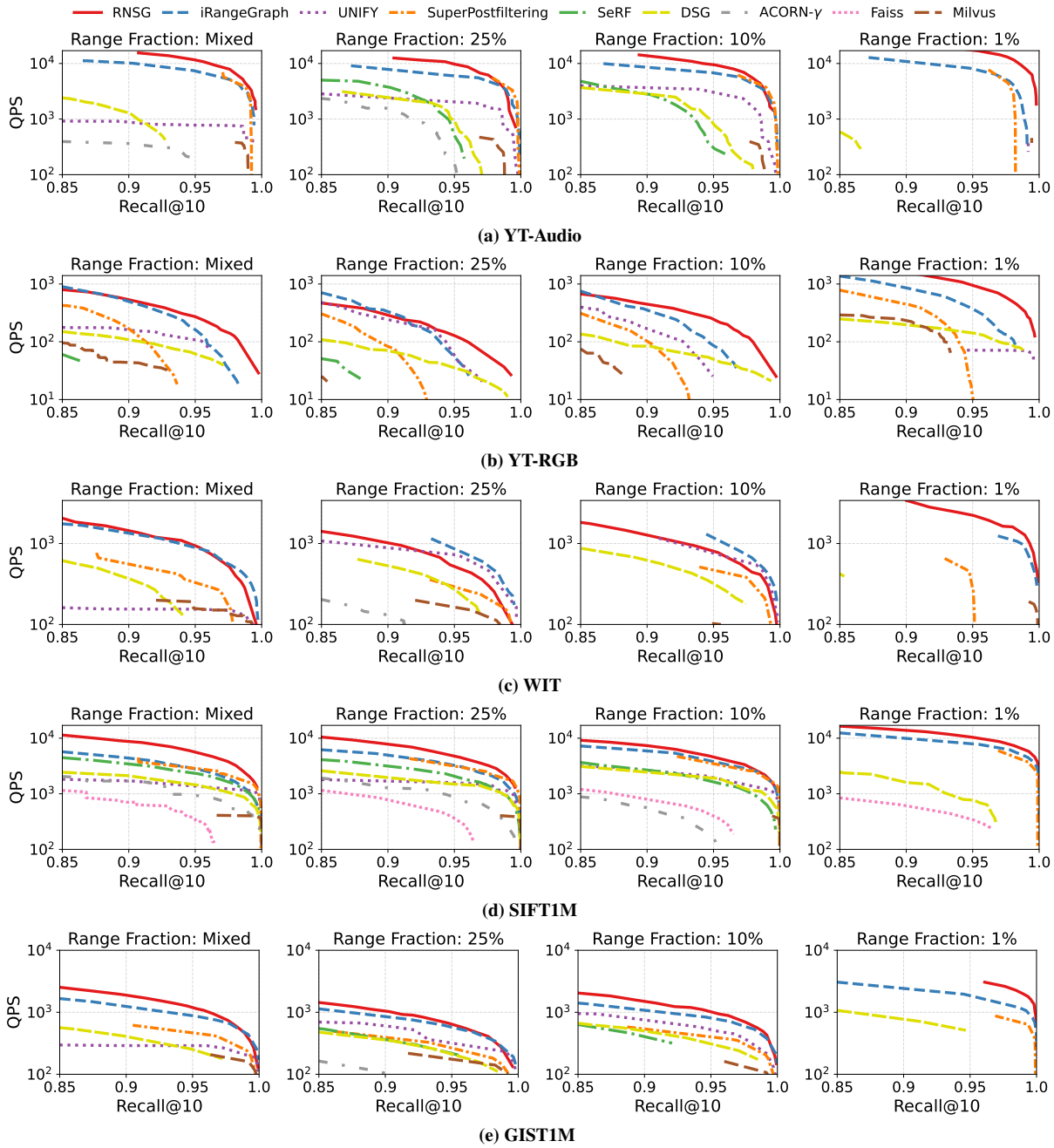


Figure 6: Query performance of different methods under varying selectivity levels (upper and right is better, QPS/Recall values below the y-axis/x-axis range are not shown)

providing analysis with varying k values (Exp-5). For efficiency measurement, we use queries per second (QPS), calculated as QPS $= \frac{|Q|}{t}$, where $|Q|$ queries are processed within time t.

Compared Methods. We evaluate our RNSG method against 8 representative baselines for RFANN search: (1) iRangeGraph [50]: Integrates segment trees with graph indexes, constructing elemental graphs for multiple ranges and dynamically composing them during query processing (Section 2). (2) UNIFY [30]: Partitions data by numeric attributes, builds intra-segment graphs, and employs skip lists with edge bitmaps for efficient range pruning (Section 2). (3) SuperPostfiltering [13]: A post-filtering approach that pre-builds graph indexes for multiple overlapping ranges and selects the minimal covering range at query time (Section 2). (4) SeRF [54]: Employs aggressive graph compression to reduce indexing costs while maintaining search performance. We use the MaxLeap variant which showed optimal performance in the original paper. (5) DSG [39]: Designed for dynamic streaming scenarios, it uses rectangle trees for range partitioning and builds upon SeRF with optimized construction efficiency. (6) ACORN-γ [38]: A predicate-agnostic hybrid ANN

search method applicable to arbitrary filters, though less optimized for RFANN queries specifically. (7) Faiss [12]: A widely-used vector database that filters out-of-range data points during retrieval using inverted file indexing with product quantization. (8) Milvus [45]: Employs HNSW indexing with automatic strategy selection among pre-filtering, in-filtering, and post-filtering approaches based on cost estimation (Section 2). Among the evaluated methods, iRangeGraph [50] and UNIFY [30] represent the current state-of-the-art in RFANN search, as established by a recent benchmarking study [28]. We also note two recent methods, RangePQ [51] and DIGRA [25], which are designed for dynamic RFANN scenarios with efficient index updates. However, their own experiments show that for static RFANN tasks, their query performance is often inferior to representative static methods like iRangeGraph [50] and SuperPostfiltering [13] in high-dimensional vector search. Therefore, we exclude them from our comparisons.

Parameter Settings. The construction of our RNSG index involves three key parameters (Section 4.2): ef_{spatial} controls the size of the spatial-similar neighborhood, $ef_{\text{attribute}}$ bounds the size of the attribute-similar neighborhood, and m specifies the maximum final neighborhood size. We set $ef_{\text{spatial}} = 300$ across all datasets. For SIFT1M and YT-Audio, we use $m = 200$ and $ef_{\text{attribute}} = 1,500$; For WIT, we use $m = 200$ and $ef_{\text{attribute}} = 6,000$; for all other datasets, $m = 300$ and $ef_{\text{attribute}} = 10,000$. These configurations achieve an effective balance between index quality and construction efficiency in our experiments. We will investigate the sensitivity of RNSG performance to these parameters in Exp-4. For baseline methods, we employ their recommended parameter settings as provided in the original publications, ensuring optimal performance for each compared method.

Experimental Environment. All methods, including our RNSG and all baselines, are implemented in C++ and compiled with GCC 15.1.0 using -O3 optimization. Since the source codes of all baselines are publicly accessible online, we use their original implementations in our experiments. We employ Euclidean distance as the vector similarity metric throughout our experiments. We measure indexing time on our server with 128 threads and evaluate search performance using a single thread. All experiments are conducted on a server with an AMD Ryzen Threadripper 3990X Processor (2.2GHz) and 224 GB of memory.

5.2 Experimental Results

Exp-1: RFANN Querying Performance. We compare the overall RFANN query performance of our RNSG method with the baselines in Fig. 6. The main findings are summarized as follows. ***(1) Consistent Superior Performance***: RNSG consistently achieves the best performance under both mixed and fixed query workloads on most datasets. For instance, on the YT-Audio dataset at recall@10=0.98, RNSG attains a QPS of 6,656 under the mixed workload—a 1.5× speedup over the best baseline, iRangeGraph (QPS of 4,392). With a fixed query range fraction of 1%, RNSG achieves a QPS of 3,224, which is a 1.9× speedup over iRangeGraph (QPS of 1,688). ***(2) Strength on Low-Selectivity Queries***: Our method performs particularly well on low-selectivity queries. This is because RNSG preserves desirable search properties within the induced subgraph for any query ranges, while other methods face trade-offs: some prune excessive edges, leading to low recall (e.g., SeRF), while others evaluate numerous out-of-range objects, reducing efficiency (e.g., iRangeGraph). For example, on SIFT1M, when selectivity equals 1%, RNSG achieves recall@10 = 1.0 with QPS = 2,122, while iRangeGraph can only get recall@10 = 0.99 with QPS = 614, reaching a 3.5× speedup with superior accuracy. ***(3) Robustness in High-Recall Scenarios***: RNSG demonstrates substantially superior performance under high-recall requirements (e.g., recall@10 $\geq$ 0.98). While SOTA methods like iRangeGraph and UNIFY must expand the search scope significantly to achieve a high recall, incurring high computational cost, our method preserves efficiency due to the inherent hereditary property and high searchability of the RNSG structure. For instance, on the YT-RGB dataset at recall@10 = 0.98, RNSG (QPS=1,110) achieves a 2.7× higher QPS than iRangeGraph (QPS=412). ***(4) Baseline Limitations***: We observe two key limitations in the baselines. First, methods not designed for RFANN queries, including ACORN-γ, Faiss, and Milvus, exhibit extremely low QPS (most QPS values of these methods are below the y-axis range and therefore do not appear on the figure). Second, SeRF's aggressive compression technique (leads to a low-quality index graph) prevents it from achieving high recall, often failing to reach 0.85 on many datasets. Both findings align with previous studies [28, 50]. These results demonstrate the superiority of our RNSG method for RFANN search, achieving high efficiency and robust performance.

Exp-2: Index Construction Time. Fig. 7 compares the index construction time of all methods across 5 datasets. The results demonstrate that our RNSG achieves an excellent balance between construction efficiency and query performance. On the SIFT1M dataset, RNSG requires only 78.12 seconds to build the index, while the best-performing baselines need significantly more time: iRangeGraph takes 467.15 seconds, UNIFY requires 1168 seconds, and SuperPostFiltering needs 1345.4 seconds. This represents at least 6× faster construction compared to state-of-the-art methods. When compared with construction-efficient approaches like Milvus, RNSG maintains similar build times on most dataset (1349.64 seconds vs. 823.14 seconds for Milvus on WIT) while achieving substantially better query performance (delivering 3.7× higher QPS at recall@10 = 0.98). We also observe that while SeRF constructs indexes quickly, its query performance is considerably worse than RNSG on all datasets, particularly for queries with low selectivity, as shown in Fig. 6.

Exp-3: Index Size. Fig. 8 presents the memory overheads (index size) of all indexing methods across 5 datasets. As can be seen, our RNSG consistently demonstrates much lower memory consumption compared to the best-performing methods (iRangeGraph, UNIFY, and SuperPostFiltering). For example, on the YT-RGB dataset, our proposed RNSG consumes only 223.83 MB, achieving a 5.4× reduction compared with the best-performing baseline iRangeGraph, which requires 1218.78 MB. When compared with memory-efficient approaches like Faiss, RNSG maintains similar memory overhead (223.83 MB vs. 268.42 MB) while delivering substantially superior query performance (achieving 8.6× speedup at recall@10 = 0.8). Consistent with our indexing time analysis, we observe that SeRF achieves the lowest memory footprint among all methods due to its aggressive graph compression strategy. However, this space efficiency comes at the substantial cost of query performance degradation, as shown in Fig. 6. These results further highlights the superiority of our RNSG approach, which maintains competitive memory usage while delivering state-of-the-art search performance.

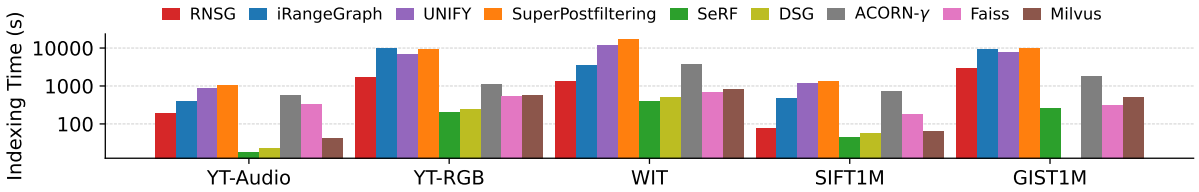


Figure 7: Index construction time of different methods on all datasets

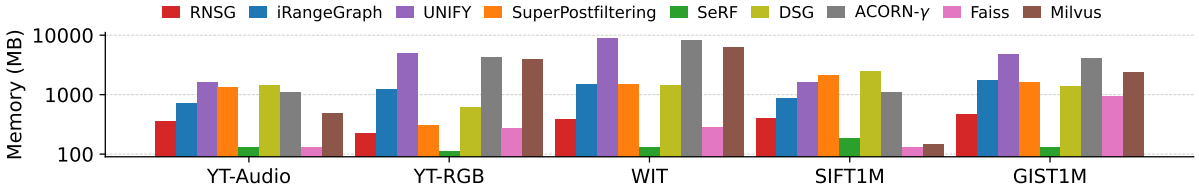
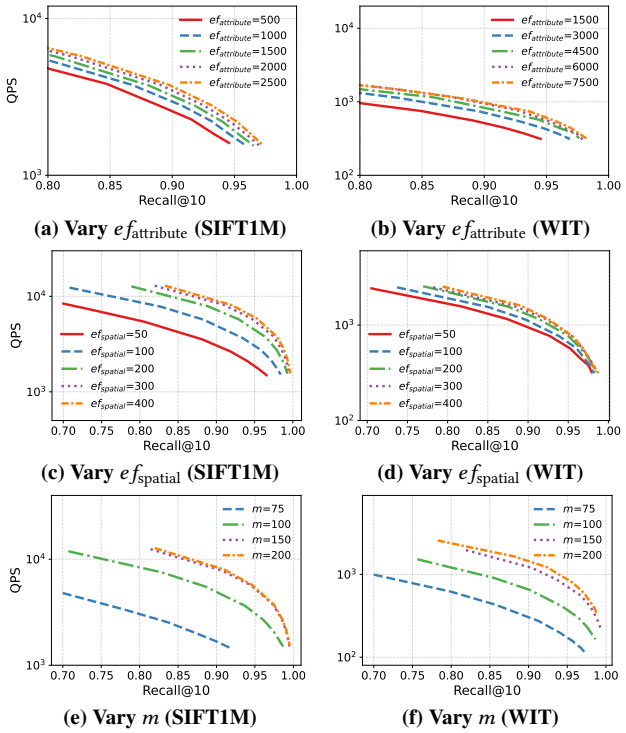


Figure 8: Index size of different methods on all datasets

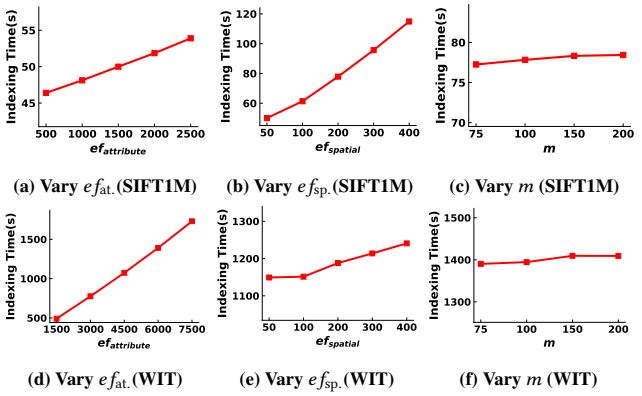


(a) Vary $ef_{\text{attribute}}$ (SIFT1M) (b) Vary $ef_{\text{attribute}}$ (WIT)

(c) Vary ef_{spatial} (SIFT1M) (d) Vary ef_{spatial} (WIT)

(e) Vary m (SIFT1M) (f) Vary m (WIT)

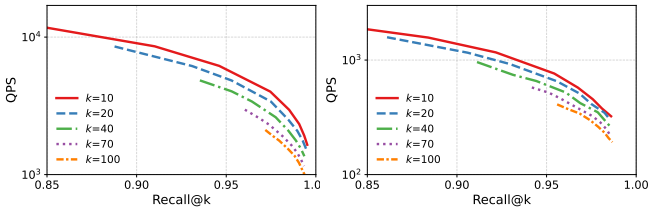
Figure 9: Impact of Parameters on RNSG Query Performance



(a) Vary $ef_{\text{at.}}$(SIFT1M) (b) Vary $ef_{\text{sp.}}$(SIFT1M) (c) Vary m (SIFT1M)

(d) Vary $ef_{\text{at.}}$(WIT) (e) Vary $ef_{\text{sp.}}$(WIT) (f) Vary m (WIT)

Figure 10: Impact of Parameters on RNSG Indexing Time



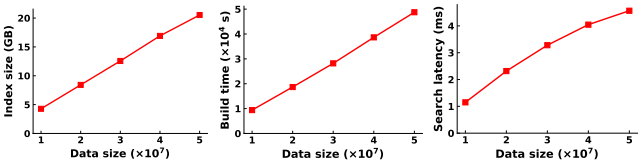
(a) SIFT1M (b) WIT

Figure 11: Query Performance of RNSG with Varying k Values

Exp-4: Parameter Sensitivity Analysis. This experiment investigates the impact of parameters $ef_{\text{attribute}}$, ef_{spatial}, and m on RNSG query performance. Fig. 9 shows query performance with varying parameters on SIFT1M and WIT datasets, with consistent trends observed on other datasets. Query performance improves with increasing values of all three parameters, as larger parameter values enable consideration of more candidate neighbors, resulting in higher-quality graph connectivity and enhanced searchability. However, the performance gains exhibit diminishing returns, i.e., the marginal improvement decreases with larger parameter values. Given that larger parameters also increase indexing costs, we recommend selecting moderate parameter values that achieve an optimal balance between query performance and construction overhead. For example, on SIFT1M, performance gains become marginal beyond $ef_{\text{attribute}} = 1,500$, $ef_{\text{spatial}} = 200$, and $m = 200$, justifying our selection of these as default values in our previous experiments.

Fig. 10 shows that the construction time grows smoothly with $ef_{\text{attribute}}$ and m, but it exhibits a slightly sharper increase with ef_{spatial}. This differential sensitivity arises because ef_{spatial} directly affects the NNDescent algorithm [11] for initial KNN graph construction (Algorithm 2), which exhibits higher parameter sensitivity, while our range-aware pruning strategy remains efficient and less sensitive to $ef_{\text{attribute}}$ and m. Notably, indexing time stabilizes as m approaches 200 on both SIFT1M and WIT, further validating our default parameter selection.

Exp-5: Results with Various k. This experiment evaluates RNSG's query performance across different k values. Fig. 11 presents results on SIFT1M and WIT, with consistent results observed on other



(a) Index size (b) Indexing time (c) Latency at 0.9 recall

Figure 12: Scalability of RNSG on SIFT

datasets. As expected, QPS decreases as k increases due to the expanded search scope required to retrieve more results. Concurrently, recall improves significantly with larger k values, as the beam search parameter ef_{search} must exceed k to ensure sufficient candidate exploration. This expanded search space naturally enhances result quality while increasing processing time. The QPS degradation remains gradual across the k spectrum, demonstrating RNSG's robust performance to varying retrieval requirements. These results further validate the efficiency and effectiveness of our proposed solution across various scenarios.

Exp-6: Scalability. This experiment evaluates RNSG's scalability using SIFT datasets[7] ranging from 10 to 50 million objects. We maintain the default parameters established for SIFT1M and employ the mixed query workload described in our experimental setup. Results in Fig. 12 show that both indexing time and memory consumption scale linearly with dataset size. Query performance degrades smoothly with increasing data volume, exhibiting logarithmic square complexity characteristics. These findings demonstrate RNSG's strong scalability in both index construction and query processing, confirming its suitability for large-scale applications.

Summary and Recommendation. In terms of index size, RNSG generates significantly more compact indexes than specialized RFANN methods (iRangeGraph, UNIFY, SuperPostfiltering, DSG) and general-purpose systems (ACORN-γ, Milvus), while matching the efficiency of Faiss. Only SeRF produces smaller indexes, but this comes at the cost of severely degraded query performance due to aggressive compression. For indexing time, RNSG substantially outperforms iRangeGraph, UNIFY, SuperPostfiltering, and ACORN-γ, while achieving comparable construction speed to DSG, Faiss, and Milvus. Although SeRF builds indexes faster, its poor query performance limits its practical utility. Most importantly, in query performance, RNSG consistently outperforms all baselines across diverse selectivity levels on most datasets, with particularly notable advantages for low-selectivity queries. Methods like ACORN-γ, Faiss and Milvus show limited effectiveness due to their non-specialized designs for RFANN queries, while SeRF and DSG suffer from performance degradation caused by excessive graph compression.

Table 2 summarizes our recommendations using a star-rating system. For practical RFANN deployment, RNSG represents the optimal choice, delivering superior query performance with reasonable indexing overhead. iRangeGraph serves as a competent alternative for high-selectivity scenarios (Fig. 6), while UNIFY and SuperPostfiltering, despite their merits, are consistently outperformed by RNSG across most evaluation scenarios.

6 Related Work

Approximate Nearest Neighbor Search. Existing ANN algorithms fall into four main categories: graph-based [5, 16, 22, 32, 33, 47], quantization-based [17, 18, 20, 24, 34], hashing-based [40, 44, 46], and tree-based methods [7, 42]. For a comprehensive overview, we refer readers to recent surveys [35, 36]. Extensive empirical evaluations [3, 5, 29, 46] have consistently shown that graph-based methods achieve state-of-the-art performance. This superiority is attributed to the graph's ability to encode proximity relationships, enabling query processing to converge rapidly by examining only a small fraction of the dataset [10]. Most graph-based indexes [47] are built upon four fundamental graph types: the Delaunay Graph (DG) [14], Relative Neighborhood Graph (RNG) [43], K-Nearest Neighbor Graph (KNNG) [37], and Minimum Spanning Tree (MST) [26]. Among these, RNG-based indexes deliver particularly high performance due to their effective pruning strategy [47]. Building upon the RNG, this work introduces a novel range-aware RNG index, specifically designed to enable efficient RFANN search.

Table 2: Summary of the performances of all methods (the more stars, the better. ☆ denotes an uncertain star ranging from 0-1)

Methods	Index Size	Indexing Time	Query Performance	Overall
iRangeGraph [50]	★★★	★★★	★★★★	★★★★
UNIFY [30]	★★☆	★★★	★★★☆	★★★
SuperPostfiltering [13]	★★★	★★★	★★★	★★★
SeRF [54]	★★★★★	★★★★★	★	★★
DSG [39]	★★★	★★★★☆	★★	★★☆
ACORN-γ [38]	★★☆	☆	☆	★
Faiss [12]	★★★★☆	★★★★☆	☆	★
Milvus [45]	★★☆	★★★★☆	★	★★
RNSG (Our method)	★★★★☆	★★★★	★★★★★	★★★★★

Attribute-filtered ANN Search. Most existing RFANN studies [12, 45, 51] follow a two-stage paradigm that separates vector and attribute retrieval before merging results. For example, ADBV [49] uses a B-Tree and a PQ index separately, choosing a strategy via a cost model. A fundamental weakness of these methods is that they do not co-optimize the index structure, resulting in performance that is non-competitive with modern hybrid indexes [8, 13, 28, 30, 50, 54]. Recent hybrid indexes like iRangeGraph [50] integrate graph and attribute indexes more directly by building a graph for each segment of a partitioned attribute range. While this yields high performance, it does so at the cost of significant indexing overhead. Beyond RFANN-specific methods, general-purpose systems like Faiss [12], Milvus [45], VBASE [52], and ACORN [38] handle filtered queries on unordered attributes. Since they do not leverage the ordering of numeric attributes in their index structures or algorithms, they are consistently outperformed by RFANN-optimized methods, a result aligned with existing findings [28, 50].

7 Conclusion

In this paper, we propose a novel graph index for efficient range-filtered approximate nearest neighbor (RFANN) search. We first establish a foundational graph indexing theory, the Range-Aware Relative Neighborhood Graph (RRNG), which integrates both spatial and attribute proximity of the data objects. The RRNG guarantees that beam search can find the exact nearest neighbor for any node, under any query range. As exact RRNG construction is computationally expensive, we propose RNSG, a practical solution that approximates the RRNG efficiently. Comprehensive experiments on five real-world datasets demonstrate that RNSG outperforms state-of-the-art methods, achieving superior index compactness, faster construction times, and significantly higher query performance. Future work includes extending our graph index to support dynamic updates and non-numerical attribute filters.

[7] http://corpus-texmex.irisa.fr/

References

[1] Akhil Arora, Sakshi Sinha, Piyush Kumar, and Arnab Bhattacharya. 2018. HD-Index: Pushing the Scalability-Accuracy Boundary for Approximate kNN Search in High-Dimensional Spaces. *Proc. VLDB Endow.* 11, 8 (2018), 906–919.

[2] Sunil Arya and David M Mount. 1993. Approximate nearest neighbor queries in fixed dimensions.. In *SODA*, Vol. 93. 271–280.

[3] Martin Aumüller, Erik Bernhardsson, and Alexander John Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Inf. Syst.* 87 (2020).

[4] Anonymous authors. [n. d.]. RNSG: A Range-Aware Graph Index for Efficient Range-Filtered Approximate Nearest Neighbor Search. In *https://anonymous.4open.science/r/R-NSG-6BE1/.*

[5] Ilias Azizi, Karima Echihabi, and Themis Palpanas. 2025. Graph-Based Vector Search: An Experimental Evaluation of the State-of-the-Art. *Proc. ACM Manag. Data* 3, 1 (2025), 43:1–43:31.

[6] Kevin Beyer, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. 1999. When is "nearest neighbor" meaningful?. In *ICDT*. 217–235.

[7] Alina Beygelzimer, Sham Kakade, and John Langford. 2006. Cover trees for nearest neighbor. In *ICML*. 97–104.

[8] Yannis Chronis, Helena Caminal, Yannis Papakonstantinou, Fatma Özcan, and Anastasia Ailamaki. 2025. Filtered Vector Search: State-of-the-Art and Research Opportunities. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5488–5492.

[9] Mark De Berg, Otfried Cheong, Marc Van Kreveld, and Mark Overmars. 2008. *Computational geometry: algorithms and applications*.

[10] Magdalen Dobson, Zheqi Shen, Guy E. Blelloch, Laxman Dhulipala, Yan Gu, Harsha Vardhan Simhadri, and Yihan Sun. 2023. Scaling Graph-Based ANNS Algorithms to Billion-Size Datasets: A Comparative Analysis. *CoRR* abs/2305.04359 (2023).

[11] Wei Dong, Moses Charikar, and Kai Li. 2011. Efficient k-nearest neighbor graph construction for generic similarity measures. In *WWW*. 577–586.

[12] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The faiss library. *arXiv preprint arXiv:2401.08281* (2024).

[13] Joshua Engels, Benjamin Landrum, Shangdi Yu, Laxman Dhulipala, and Julian Shun. 2024. Approximate Nearest Neighbor Search with Window Filters. In *ICML*.

[14] Steven Fortune. 2004. Voronoi Diagrams and Delaunay Triangulations. In *Handbook of Discrete and Computational Geometry, Second Edition*. 513–528.

[15] Cong Fu, Changxu Wang, and Deng Cai. 2021. High dimensional similarity search with satellite system graph: Efficiency, scalability, and unindexed query compatibility. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 8 (2021), 4139–4150.

[16] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474.

[17] Jianyang Gao and Cheng Long. 2024. Rabitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proc. ACM Manag. Data* 2, 3 (2024), 1–27.

[18] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized product quantization. *IEEE transactions on pattern analysis and machine intelligence* 36, 4 (2013), 744–755.

[19] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, et al. 2023. Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters. In *WWW*. 3406–3416.

[20] Yunchao Gong, Svetlana Lazebnik, Albert Gordo, and Florent Perronnin. 2012. Iterative quantization: A procrustean approach to learning binary codes for large-scale image retrieval. *IEEE transactions on pattern analysis and machine intelligence* 35, 12 (2012), 2916–2929.

[21] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, Zhenshan Cao, Yanliang Qiao, Ting Wang, Bo Tang, and Charles Xie. 2022. Manu: A Cloud Native Vector Database Management System. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3548–3561.

[22] Ben Harwood and Tom Drummond. 2016. Fanng: Fast approximate nearest neighbour graphs. In *CVPR*. 5713–5722.

[23] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *STOC*. 604–613.

[24] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.

[25] Mengxu Jiang, Zhi Yang, Fangyuan Zhang, Guanhao Hou, Jieming Shi, Wenchao Zhou, Feifei Li, and Sibo Wang. 2025. DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter. *Proc. ACM Manag. Data* 3, 3 (2025), 1–26.

[26] Joseph B Kruskal. 1956. On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem. *Proceedings of the American Mathematical society* 7, 1 (1956), 48–50.

[27] Jie Li, Haifeng Liu, Chuanghua Gui, Jianyu Chen, Zhenyuan Ni, Ning Wang, and Yuan Chen. 2018. The design and implementation of a real time visual search system on JD E-commerce platform. In *Proceedings of the 19th International Middleware Conference Industry*. 9–16.

[28] Mocheng Li, Xiao Yan, Baotong Lu, Yue Zhang, James Cheng, and Chenhao Ma. 2026. Attribute Filtering in Approximate Nearest Neighbor Search: An In-depth Experimental Study. *Proc. ACM Manag. Data* (2026).

[29] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2020. Approximate Nearest Neighbor Search on High Dimensional Data — Experiments, Analyses, and Improvement. *IEEE Transactions on Knowledge and Data Engineering* 32, 8 (2020), 1475–1488.

[30] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *Proc. VLDB Endow.* 18, 4 (2024), 1118–1130.

[31] Ying Liu, Dengsheng Zhang, Guojun Lu, and Wei-Ying Ma. 2007. A survey of content-based image retrieval with high-level semantics. *Pattern recognition* 40, 1 (2007), 262–282.

[32] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68.

[33] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2018), 824–836.

[34] Yusuke Matsui, Yusuke Uchida, Hervé Jégou, and Shin'ichi Satoh. 2018. A survey of product quantization. *ITE Transactions on Media Technology and Applications* 6, 1 (2018), 2–10.

[35] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *VLDB J.* 33, 5 (2024), 1591–1615.

[36] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Vector Database Management Techniques and Systems. In *Companion of the 2024 International Conference on Management of Data*. 597–604.

[37] Rodrigo Paredes and Edgar Chávez. 2005. Using the *k*-Nearest Neighbor Graph for Proximity Searching in Metric Spaces. In *SPIRE (Lecture Notes in Computer Science, Vol. 3772)*. 127–138.

[38] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. Acorn: Performant and predicate-agnostic search over vector embeddings and structured data. *Proc. ACM Manag. Data* 2, 3 (2024), 1–27.

[39] Zhencan Peng, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2025. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 18, 10 (2025), 3256–3268.

[40] Ninh Pham and Tao Liu. 2022. Falconn++: A locality-sensitive filtering approach for approximate nearest neighbor search. *Advances in Neural Information Processing Systems* 35 (2022), 31186–31198.

[41] Jeffrey Pound, Floris Chabert, Arjun Bhushan, Ankur Goswami, Anil Pacaci, and Shihabur Rahman Chowdhury. 2025. MicroNN: An On-device Disk-resident Updatable Vector Database. In *Companion of SIGMOD*. 608–621.

[42] Parikshit Ram and Kaushik Sinha. 2019. Revisiting kd-tree for nearest neighbor search. In *KDD*. 1378–1388.

[43] Godfried T Toussaint. 1980. The relative neighbourhood graph of a finite planar set. *Pattern recognition* 12, 4 (1980), 261–268.

[44] Jingdong Wang, Heng Tao Shen, Jingkuan Song, and Jianqiu Ji. 2014. Hashing for similarity search: A survey. *arXiv preprint arXiv:1408.2927* (2014).

[45] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A purpose-built vector data management system. In *SIGMOD*. 2614–2627.

[46] Jingdong Wang, Ting Zhang, Nicu Sebe, Heng Tao Shen, et al. 2017. A survey on learning to hash. *IEEE transactions on pattern analysis and machine intelligence* 40, 4 (2017), 769–790.

[47] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 14, 11 (2021), 1964–1978.

[48] Roger Weber, Hans-Jörg Schek, and Stephen Blott. 1998. A Quantitative Analysis and Performance Study for Similarity-Search Methods in High-Dimensional Spaces. In *VLDB*. 194–205.

[49] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. Analyticdb-v: A hybrid analytical engine towards query fusion for structured and unstructured data. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3152–3165.

[50] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S Jensen. 2024. irangegraph: Improvising range-dedicated graphs for range-filtering nearest neighbor search. *Proc. ACM Manag. Data* 2, 6 (2024), 1–26.

[51] Fangyuan Zhang, Mengxu Jiang, Guanhao Hou, Jieming Shi, Hua Fan, Wenchao Zhou, Feifei Li, and Sibo Wang. 2025. Efficient Dynamic Indexing for Range Filtered Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 3 (2025), 1–26.

[52] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, et al. 2023. {VBASE}: Unifying online vector similarity search and relational queries via relaxed monotonicity. In *OSDI*. 377–395.

[53] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. 2024. Retrieval-augmented generation for ai-generated content: A survey. *arXiv preprint arXiv:2402.19473* (2024).

[54] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: segment graph for range-filtering approximate nearest neighbor search. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–26.

A Appendix

A.1 Proof of Theorem 3.6

LEMMA A.1 (EXPECTED LENGTH OF MONOTONIC PATH). *For any start $x \in V$ and the interval-wise nearest neighbor $y \in V$ of a query q, the greedy monotone walk on G from x to y has expected length*

$$\mathbb{E}\big[\text{steps on } G\big] = O\left(\frac{n^{1/d}\,\log n}{\Delta r}\right), \qquad n := |V|.$$

, where Δr denotes the minimum difference in side lengths among any non-isoscele triangles in the space. Moreover, if x and y are label-adjacent in I (no label strictly between $a(x)$ and $a(y)$), then $(x, y) \in E[G[I]]$ and the walk terminates in one step.

PROOF. Since RRNG exhibits perfect monotonicity, the NSG paper refers to such graphs as MSNETs. According to our theory, RNSG belongs to the class of MSNETs. Therefore, the expected path length on RNSG is at most equivalent to that of MSNETs, namely $O(n^{1/d} log(n^{1/d})/\Delta r)$ □

LEMMA A.2. *Let the average degree of RNG be D_r[16], then average degree of RRNG is $2D_r[1 + log(\frac{N-1}{2D_r})]$, note that D_r is bounded by the max degree of RNG, which is a constant in E^d and independent of n.*

PROOF. Fix $x \in V_I$ and cover the unit sphere around x by N_d cones $\{C_j\}_{j=1}^{N_d}$ of half-angle 30°. Split candidates into the *left* side $\{a(\cdot) < a(x)\}$ and the *right* side $\{a(\cdot) > a(x)\}$. For a fixed cone C and a fixed side $s \in \{\text{left}, \text{right}\}$, let M be the number of other points of V_I that fall in (C, s). By (i)–(ii) and symmetry, $M \sim \text{Bin}(n_I - 1, p)$ with $p = \frac{1}{2N_d}$.

Under RRNG pruning, within (C, s) we scan candidates in *label order*. Because s fixes the label direction (left/right), any earlier-scanned point is label-between a later one; and in a cone of half-angle 30°, if z is no farther from x than y, then xy is the longest side in the triangle (x, y, z), hence z witnesses the removal of xy. Therefore, an edge (x, y) is *kept* iff y is a *record minimum* of the distance sequence w.r.t. the scanning order. Let R_M be the number of record minima among M i.i.d. items under a random permutation independent of the distances (by (ii)); then

$$\mathbb{E}[R_M \mid M = m] \;=\; H_m \;\triangleq\; \sum_{k=1}^{m} \frac{1}{k} \quad (m \geq 0,\ H_0 := 0),$$

a classical fact proved by $\Pr\{\text{the } k\text{-th is a new minimum}\} = 1/k$ and linearity of expectation.

By linearity across cones and sides,

$$\mathbb{E}\big[\,\deg_G^+(x)\big] \;=\; \sum_{s\in\{\text{L,R}\}} \sum_{j=1}^{N_d} \mathbb{E}[R_M] \;=\; 2N_d\,\mathbb{E}[H_M].$$

For an upper bound that is closed-form and distribution-free in M, use $H_m \leq 1 + \log(1 + m)$ for all $m \geq 0$ and the concavity of log:

$$\mathbb{E}[H_M] \;\leq\; \mathbb{E}[1{+}\log(1{+}M)] \;\leq\; 1{+}\log\big(1{+}\mathbb{E}[M]\big) \;=\; 1{+}\log\left(1{+}\tfrac{n_I-1}{2N_d}\right).$$

Multiplying by $2N_d$ yields the claimed bound.

□

PROOF. Based on Lemma A.1 and Lemma A.2, we can multiply them to obtain the expected complexity of the search as

$$O(2D_r n^{1/d} \log(n^{1/d})(1 + \log(\frac{N-1}{2D_r}))/\Delta r).$$

According to prior work, Δr is approximately $O(n^{-\frac{\epsilon}{d}})$, where $0 < \epsilon \ll d$. Furthermore, D_r can be regarded as a constant. Thus, the formula can be simplified to

$$O(n^{-\frac{\epsilon}{d}}/\Delta r).$$

Based on prior work[16], Δr is approximately $O(n^{-\epsilon/d})$, where $0 < \epsilon < d$. Since D_r can be treated as a constant, the formula can be simplified to $O(n^{\frac{d+\epsilon}{d}} \log^2 N)$. Given that $\epsilon \ll d$, we can ultimately approximate this as $O(n^{1/d} \log^2 n)$, thus the theorem is established. □

A.2 Proof of Theorem 3.7

PROOF. The space complexity of RRNG consists of two parts: the number of nodes and the number of edges. The number of nodes is $O(n)$. According to Lemma A.2, we can derive that the number of edges in RRNG is $O(2nD_r[1 + \log(\frac{N-1}{2D_r})])$. Furthermore, since D_r is the average number of edges for MRNG, the expected number of edges for MRNG is $S_r = nD_r$. Thus, the number of edges can be expressed as

$$O(2S_r[1 + \log(\frac{N-1}{2D_r})]).$$

Simplifying this yields $O(S_r \log n)$. Since the number of edges in this expression far exceeds the number of vertices, we can treat the edge count and space complexity as equivalent. Therefore, the space complexity is also $O(S_r \log n)$. □

A.3 Proof of Theorem 4.3

PROOF. After sorting all nodes by attribute, we select the ith node v_i to apply the pruning result. According to Lemma 4.1, the pruning results for any set of points with $j < i$ do not interfere with those for any set with $j > i$. Without loss of generality, we consider the case where Algorithm 1 is applied to all points v_j with $j > i$ separately. By Lemma 4.2, for any v_j, we only need to check whether any point v_k with $i < k < j$ triggers the pruning condition. Since our pruning algorithm traverses the entire set in order of $|v_j.a - v_i.a|$, which is equivalent to traversing in ascending order of j, we can apply mathematical induction:

Base Case: For the nearest candidate v_{j+1}, no other vertex k satisfies $j < k < j + 1$, so it must appear in the neighborhood set. Furthermore, by the definition of RRNG, it must also belong to RRNG's neighborhood set.

Inductive Step: Suppose we have applied the pruning algorithm to all $i < k < j$ and obtained the true neighbor sets. We then consider two cases:

Case 1: If the pruning algorithm determines that the current edge (v_i, v_j) does not exist, then there must be a vertex v_k already in the neighborhood set such that $\delta(v_i, v_j) > \delta(v_i, v_k)$ and $\delta(v_i, v_j) > \delta(v_i, v_k)$. Since v_k is already in the neighborhood set, the edge $(v_i, v_k) \in E$ must exist in the true RRNG graph. Furthermore, since our scanning order proceeds from $i + 1$ to j, k is a previously checked vertex, so $i + 1 \leq k < j$. Thus, $v_i.a < v_k.a < v_j.a$ satisfies the pruning condition in the RRNG definition, meaning this edge also does not exist in the RRNG.

Case 2: If the pruning algorithm determines that the current edge (v_i, v_j) exists, it indicates that none of the previously stored neighbors satisfy $\delta(v_i, v_j) > \delta(v_i, v_k)$ and $\delta(v_i, v_j) > \delta(v_i, v_k)$, since the previously stored neighbor set matches the actual set. This implies that for $i < k < j$, v_i has no neighbor v_k in RRNG that

satisfies the triangle condition and $\delta(v_i, v_j) > \delta(v_i, v_k)$, violating the definition. Therefore, the edge (v_i, v_j) is also retained in the actual RRNG. By induction, the set obtained by the pruning algorithm equals the true neighbor set in the RRNG. □

A.4 Proof of Theorem 4.7

Proof. For any query interval $I = [a_l, a_r]$, let $G = (V, E)$ be the RNSG on the full dataset with KNNG G', and $G[I] = (V_I, E')$ be the subgraph induced by nodes in I, where V_I denotes the node subset filtered by the range I. Let $H[I] = (V_I, E_I)$ be the RNSG constructed directly on V_I with the induced graph of G'. Since V_I is common to both, we prove $E' = E_I$ by demonstrating that for any given vertex, its neighborhood sets are identical in both graphs.

Let $G = (V, E)$ be an RNSG on D under metric δ, built with the $ef_{spatial}$-NN graph G' and RAP (Alg. 1) with per-node budget m. For any interval $I = [a_l, a_r]$, let $G[I] = (V_I, E')$ be the vertex-induced subgraph on $V_I = \{x \in V : x.a \in I\}$. Let $H[I] = (V_I, E_I)$ be the RNSG constructed directly on V_I using the induced $ef_{spatial}$-NN graph $G'[I]$ and the same 1.

Since V_I is common to both, now we prove $E' = E_I$.

Step 1: Same candidates after intersection. For any $x \in V$, the candidate set on V is

$$C(x) = \text{Neighbour}_{G'}(x) \cup \{v_j \mid |j - \text{rank}(x)| \le ef_{attribute}\}.$$

On V_I, the candidate set is

$$C_I(x) = \text{Neighbour}_{G'[I]}(x) \cup \{v_j \mid |j - \text{rank}(x)| \le ef_{attribute},\ v_j \in V_I\}.$$

Since $G'[I]$ is the vertex-induced subgraph of G', we have $\text{Neighbour}_{G'[I]}(x) = \text{Neighbour}_{G'}(x) \cap V_I$. Hence

$$C_I(x) = C(x) \cap V_I. \tag{1}$$

Step 2: Two sides are independent, and in-interval nodes come first. According to Lemma 4.1 RAP (Alg. 1) splits $C(x)$ into

$$C_l(x) = \{v \in C(x) : v.a < x.a\}, \quad C_r(x) = \{v \in C(x) : v.a > x.a\},$$

processes each side in nondecreasing $|v.a - x.a|$, and uses budget $m/2$ per side.

Fix $x \in V_I$. On the right side, if $y \in C_r(x) \cap V_I$ and $z \in C_r(x)$ with $|z.a - x.a| < |y.a - x.a|$, then $x.a < z.a < y.a \le a_r$, so $z \in V_I$. Thus any candidate processed before an in-interval right-side candidate is also in the interval. The left side is symmetric.

Step 3: Induction on each side (right side shown). Order $C_r(x)$ as $u_1, u_2, \ldots$ by $|v.a - x.a|$. Let

$$R'_r(x) = \big(\text{right-side kept set built on } V\big) \cap V_I,$$

$$R^I_r(x) = \text{right-side kept set built on } V_I.$$

We prove $R'_r(x) = R^I_r(x)$ for every prefix.

Base case. If $u_1 \notin V_I$, then by Step 2 we have $C_r(x) \cap V_I = \varnothing$, so both sides are empty. If $u_1 \in V_I$, there is no earlier witness and both sides share budget $m/2$, so the decision on u_1 is the same. Thus $R'_r(x) = R^I_r(x)$ initially.

Inductive step. Assume $R'_r(x) = R^I_r(x)$ after u_{t-1}. For u_t:

- If $u_t \notin V_I$, it never appears in $R^I_r(x)$; and by Step 2 it cannot serve as a witness for any later in-interval candidate. After restricting to V_I, its effect vanishes, so the equality remains.
- If $u_t \in V_I$, its decision uses only the RRNG test

$$\delta(x, w) < \delta(x, u_t) \text{ and } \delta(u_t, w) < \delta(x, u_t) \quad (\forall w \in \text{kept set}),$$

and the per-side budget $m/2$. By the induction hypothesis the kept sets are the same before testing u_t; by Step 2 all earlier witnesses are in V_I, so the budget is reached at the same time. Therefore both decisions match, and the equality holds.

Thus $R'_r(x) = R^I_r(x)$ on the right; the left side is identical.

Step 4: Union. For any $x \in V_I$, the neighbor set is the union of the two sides. Since each side matches, the unions match, so $E' = E_I$. This proves the theorem.

□

A.5 Proof of Lemma 4.8

Proof. Let $V = \langle v_1, \ldots, v_n \rangle$ be sorted in non-decreasing attribute order as in Algorithm 3, and fix a right endpoint a_r. Let v_r be the node with the largest attribute value not exceeding a_r. For each $t \le r$, define the query range $I_t = [v_t.a, a_r]$ and let c denote the centroid of V, with $dis_i \triangleq \delta(v_i, c)$.

Under the independence assumption between embeddings and attributes, the sequence $(dis_1, \ldots, dis_r)$ consists of independent and identically distributed (i.i.d.) random variables (ties occur with probability zero or are broken arbitrarily). The entry node for range I_t is defined as:

$$e_t \triangleq \arg\min_{1 \le s \le t} d_s.$$

As t increases from 1 to r, e_t changes precisely when $dis_t < \min_{s<t} dis_s$, i.e., when dis_t is a new record minimum.

For i.i.d. continuous random variables, the probability that dis_t is a new record minimum is $1/t$. Therefore, by linearity of expectation:

$$\mathbb{E}[\#\{e_t : 1 \le t \le r\}] = \sum_{t=1}^{r} \frac{1}{t} = H_r \le 1 + \ln r = O(\log n).$$

This completes the proof. □